

Implementatieplan: Adult Task Manager App

Versie 1.0 — volledig, release-ready

Inhoudsopgave

1. [Productvisie & freemium model](#)
 2. [Tech stack & motivatie](#)
 3. [Projectstructuur & architectuurprincipes](#)
 4. [Fase 1 – Foundation \(week 1–2\)](#)
 5. [Fase 2 – Core app \(week 3–5\)](#)
 6. [Fase 3 – Notificatiesysteem \(week 6–7\)](#)
 7. [Fase 4 – Multi-user & huishoudens \(week 8–10\)](#)
 8. [Fase 5 – Monetisatie \(week 11–12\)](#)
 9. [Fase 6 – Release prep \(week 13–16\)](#)
 10. [Veiligheidschecklist](#)
 11. [Volledige mappenstructuur](#)
 12. [Database schema met RLS](#)
-

1. Productvisie & freemium model

Kernidee

Een huishoud-taakmanager speciaal ontworpen voor de **Nederlandse markt**. De app begrijpt de Nederlandse context: afvalinzameling per gemeente, Nederlands weer, Nederlandse seizoenen en typisch Nederlandse huishoudtaken. In plaats van een generieke internationale app biedt dit iets wat Tody, Chap of Home Tasker niet kunnen: een app die weet wanneer jouw kliko in Culemborg geleegd wordt, en die de weersvoorspelling meeneemt bij het plannen van het ramen zemen.

Nederlandse differentiatie — de drie pijlers

1. Afvalkalender-integratie Via api.mijnafvalwijzer.nl haal je per postcode + huisnummer de ophaaldata op voor restafval, GFT, papier en PMD. Gebruiker voert eenmalig postcode + huisnummer in → app weet automatisch wanneer welke kliko naar buiten moet. Notificaties: "Morgen wordt je groene bak geleegd. Zet hem voor 07:30 buiten."

Dekking: mijnafvalwijzer.nl dekt het overgrote deel van Nederlandse gemeenten. Voor gemeenten die via Ximmio/AVRI werken (Culemborg, Tiel, Buren, etc.) is er een aparte Ximmio-endpoint als fallback. Bij onboarding detecteert de app automatisch welke provider jouw gemeente gebruikt.

2. Weer-intelligente planning (Open-Meteo API) De app gebruikt Open-Meteo (gratis, geen API key, hoge nauwkeurigheid voor NL) om weersomstandigheden mee te nemen in taakplanning. Voorbeelden:

- "Komende week zijn dinsdag en donderdag droog en zonnig — goed moment om ramen te zemen. Woensdag wordt het 8mm regen, dat sla je beter over."
- "Dit weekend is het mooi weer — misschien een goed moment om de tuin bij te houden?"

- Automatisch verschuiven van buiten-taken als er regen verwacht wordt op de geplande dag.

Relevante weersparameters: neerslag (mm), temperatuur, windsnelheid (boven 7 Bft: buiten taken uitstellen), zonneschijn. Geen complexe AI nodig — simpele drempelwaarden in de Edge Function zijn genoeg.

3. Nederlandse taakcontext Preset-taken die in internationale apps niet bestaan:

- Grofvuil aanmelden bij gemeente (via gemeentewebsite-link)
- Verwarmingsketel ontluchten (typisch NL cv-ketel)
- Dakgoot reinigen (najaar)
- Fiets onderhouden / banden oppompen
- Energienota controleren (maandelijks)
- Huurcontract/VvE-bijdrage checken (jaarlijks)

Freemium splits

Gratis (Free tier)

Taak	Standaard frequentie
Stofzuigen	Wekelijks
Badkamer schoonmaken	Wekelijks
WC schoonmaken	Wekelijks
Keuken aanrecht	Dagelijks
Vloer dweilen	Tweewekelijks
Stof afnemen	Tweewekelijks
Vaatwasser uitruimen	Dagelijks
Was doen	Wekelijks

Gratis gebruikers krijgen ook:

- Maximaal 3 eigen custom taken
- Taakgeschiedenis laatste 30 dagen
- Solo gebruik (geen huishouden)
- Basis push-notificaties

Premium (€3,49/maand of €27,99/jaar)

Taak / Feature	Toelichting
 Kliko buiten zetten	Automatisch via afvalkalender API
 Weer-intelligente planning	Open-Meteo koppeling voor buiten-taken
☐ Ramen zemen	Droog-weer reminder op basis van weersvoorspelling
 Planten water geven	Aanpasbaar interval

Taak / Feature	Toelichting
 Verwarmingsketel ontlichten	Elk half jaar, herfst-reminder
 Auto APK	Jaarlijks, 4 weken van tevoren
 Dakgoot reinigen	Najaarsherinnering
 Fiets onderhouden	Maandelijks
 Energienota controleren	Maandelijks
 Multi-user huishoudens	Tot 5 bewoners, gedeelde taken
 Statistieken & streaks	Onbeperkte geschiedenis
 Snooze & herhaal-opties	Per notificatie aanpasbaar

Prijsstrategie

De gratis tier bedient solo binnenshuis-gebruik volledig. Premium voegt de "slimme" Nederlandse context toe — dingen die je pas mist als je ze vergeet. De afvalkalender-integratie is bewust premium: het is het sterkste onderscheidende feature en heeft een duidelijke directe waarde ("nooit meer een vergeten klike").

Vergelijking met de markt: Tody kost €0.99–€6.99 eenmalig, zonder abonnement. Onze €3,49/maand is alleen gerechtvaardigd door de live data-integraties (afvalkalender + weer) die continu waarde leveren. Zorg dat die integraties vlekkeloos werken vóór je monetiseert.

2. Tech stack & motivatie

Frontend

Technologie	Versie	Reden
Expo SDK	52+	Managed workflow, EAS Build, OTA updates
React Native	0.76+	Cross-platform iOS + Android
expo-router	4.x	File-based routing, deep links, type-safe navigatie
NativeWind	4.x	Tailwind in React Native, consistent design system
React Query (TanStack)	5.x	Server state, caching, optimistic updates
Zustand	5.x	Lichte UI-state (filters, modals)
Zod	3.x	Runtime validatie van formulieren en API-responses
expo-secure-store	latest	Veilige opslag van auth tokens (niet AsyncStorage)
expo-notifications	latest	Push token beheer, lokale notificaties

Backend

Technologie	Reden
Supabase	Auth, PostgreSQL, Edge Functions, Realtime, Storage — alles in één
Row Level Security	Beveiliging direct in de database, niet alleen in de app
Supabase Edge Functions	Deno-based serverless functies voor cron jobs
Supabase Vault	Veilige opslag van secrets in Edge Functions

Externe data-integraties (NL-specifiek)

Service	Gebruik	Kosten
api.mijnaafvalwijzer.nl	Afvalophaaldata per postcode + huisnummer. Dekkt het merendeel van NL gemeenten. Geen officiële API key nodig (zelfde key als de consumentenapp).	Gratis
Ximmio Waste Platform	Fallback voor AVRI-regio (Culemborg, Tiel, Buren, etc.) via wasteportal.ximmio.com . Zelfde postcode/huisnummer interface.	Gratis
Open-Meteo	Weersvoorspelling voor weer-intelligente taakplanning. Volledig gratis, geen API key, hoge nauwkeurigheid voor NL. Endpoint: api.open-meteo.com/v1/forecast .	Gratis

Let op: api.mijnaafvalwijzer.nl is een onofficiële API (reverse-engineered uit de consumentenapp). Dit is een bewuste risico-afweging: de API is al jaren stabiel en wordt door duizenden Home Assistant-installaties gebruikt. Houd een fallback klaar (handmatige invoer van ophaaldag) voor het geval de API wijzigt. Bij schaal > 10.000 gebruikers overweeg je een eigen scraper of contact met Ximmio/RD4 voor een officiële samenwerking.

Monetisatie

Technologie	Reden
RevenueCat	Abstractielaag over Apple IAP + Google Play Billing. Handles receipt validation, webhooks, subscriber status. Vermijdt zelf implementeren van App Store logica, wat complex en foutgevoelig is.

DevOps & CI/CD

Technologie	Reden
EAS Build	Cloud builds voor iOS + Android zonder lokale Xcode/Android Studio
EAS Submit	Automatisch uploaden naar App Store Connect en Google Play
GitHub Actions	CI pipeline: lint, type-check, test bij elke push
EAS Secrets	Veilige opslag van build-time environment variabelen

Monitoring

Technologie	Reden
Sentry	Crash reporting + performance monitoring in productie
PostHog	Privacy-vriendelijke analytics (GDPR-compliant, self-hostable optie)
Sentry	Crash reporting + performance monitoring
PostHog	Privacy-vriendelijke analytics (GDPR-compliant)
Uptime monitoring (bijv. BetterUptime)	Alert als de afvalkalender-sync of weer-API faalt — gebruikers missen dan notificaties zonder dat ze het weten

3. Projectstructuur & architectuurprincipes

Kernprincipes

Separation of concerns. Schermen weten niet hoe data opgehaald wordt. Hooks weten niet hoe schermen eruitzien. Services weten niet wie ze aanroept.

Type-safety van database tot component. Supabase genereert TypeScript types uit je schema. Zod valideert formulierinput. Nooit **any** in je codebase.

RLS als primaire beveiliging. Elke beveiliging in de app zelf (hide premium features, filter op user_id) is UX-beveiliging. De echte beveiliging zit in Postgres Row Level Security policies. Een kwaadwillende met een geldig access token kan alsnog niks.

Environment variables nooit hardcoden. Elke key, URL of secret staat in **.env.local** (lokaal) of EAS Secrets (build). **.env.local** staat altijd in **.gitignore**.

Optimistic updates voor kernacties. "Taak afvinken" voelt instant aan — de UI update direct, de server-call loopt op de achtergrond. Bij fout wordt de state teruggedraaid.

4. Fase 1 – Foundation (week 1–2)

4.1 Expo project aanmaken

```
npx create-expo-app@latest taskapp --template tabs
cd taskapp

# Expo router instellen
npx expo install expo-router expo-constants expo-linking expo-status-bar react-native-safe-area-context react-native-screens

# Styling
npm install nativewind tailwindcss
npx tailwindcss init

# State & data
npm install @tanstack/react-query zustand zod
```

```
# Supabase client
npm install @supabase/supabase-js expo-secure-store

# Notificaties
npx expo install expo-notifications expo-device expo-task-manager

# Monetisatie
npm install react-native-purchases

# Monitoring (later in phase 6, maar vroeg configureren)
npm install @sentry/react-native @posthog/react-native
```

4.2 TypeScript-strikte configuratie

`tsconfig.json`:

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "noImplicitAny": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true
  }
}
```

4.3 Environment variabelen

`.env.local` (nooit committen):

```
EXPO_PUBLIC_SUPABASE_URL=https://xxxx.supabase.co
EXPO_PUBLIC_SUPABASE_ANON_KEY=eyJ...
EXPO_PUBLIC_REVENUECAT_IOS_KEY=appl...
EXPO_PUBLIC_REVENUECAT_ANDROID_KEY=goog...
EXPO_PUBLIC_SENTRY_DSN=https://...
EXPO_PUBLIC_POSTHOG_KEY=phc_...
```

`expo-env.d.ts` (type-safe env vars):

```
declare global {
  namespace NodeJS {
    interface ProcessEnv {
      EXPO_PUBLIC_SUPABASE_URL: string;
      EXPO_PUBLIC_SUPABASE_ANON_KEY: string;
      EXPO_PUBLIC_REVENUECAT_IOS_KEY: string;
      EXPO_PUBLIC_REVENUECAT_ANDROID_KEY: string;
      EXPO_PUBLIC_SENTRY_DSN: string;
    }
  }
}
```

```
}  
}  
}
```

`.gitignore` toevoegen:

```
.env.local  
.env.*.local  
*.p8  
*.p12  
*.mobileprovision
```

4.4 Supabase project opzetten

1. Ga naar supabase.com → nieuw project
2. Regio: West Europe (Frankfurt) — voor GDPR en lage latency vanuit NL
3. Noteer: Project URL + anon key (public) + service role key (NOOIT in de app)
4. Zet service role key in EAS Secrets (niet in `.env.local`)

4.5 EAS configuratie

`eas.json`:

```
{
  "cli": { "version": ">= 10.0.0" },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "env": { "APP_ENV": "development" }
    },
    "preview": {
      "distribution": "internal",
      "env": { "APP_ENV": "preview" }
    },
    "production": {
      "autoIncrement": true,
      "env": { "APP_ENV": "production" }
    }
  },
  "submit": {
    "production": {
      "ios": { "appleId": "jouw@email.com", "ascAppId": "...", "appleTeamId":
"..." },
      "android": { "serviceAccountKeyPath": "./google-service-account.json" }
    }
  }
}
```

EAS Secrets instellen (via CLI):

```
eas secret:create --scope project --name SUPABASE_SERVICE_ROLE_KEY --value
"eyJ..."
eas secret:create --scope project --name EXPO_PUSH_ACCESS_TOKEN --value "..."
```

4.6 GitHub Actions CI

[.github/workflows/ci.yml](#):

```
name: CI
on: [push, pull_request]
jobs:
  lint-and-typecheck:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check
```



```

- run: npm test -- --passWithNoTests

eas-build-preview:
  needs: lint-and-typecheck
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: expo/expo-github-action@v8
      with:
        eas-version: latest
        token: ${{ secrets.EXPO_TOKEN }}
    - run: npm ci
    - run: eas build --profile preview --platform all --non-interactive

```

5. Fase 2 – Core app (week 3–5)

5.1 Navigatiestructuur (expo-router)

```

app/
├── _layout.tsx                # Root layout: Auth guard, QueryClient, providers
├── (auth)/
│   ├── _layout.tsx
│   ├── login.tsx
│   ├── register.tsx
│   └── onboarding.tsx        # Eerste keer: preset taken kiezen
├── (app)/
│   ├── _layout.tsx          # Tab navigator
│   ├── index.tsx            # Tab 1: Vandaag
│   ├── tasks/
│   │   ├── index.tsx        # Tab 2: Alle taken
│   │   ├── [id].tsx         # Taak detail + bewerken
│   │   └── add.tsx          # Nieuwe taak toevoegen
│   ├── settings/
│   │   ├── index.tsx        # Tab 3: Instellingen
│   │   ├── household.tsx    # Huishouden beheren [Premium]
│   │   ├── subscription.tsx # Abonnement & upgrade
│   │   └── account.tsx      # Account, data-export, verwijderen
└── +not-found.tsx

```

Root layout `app/_layout.tsx`:

```

import { useEffect } from 'react';
import { Slot, useRouter, useSegments } from 'expo-router';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { useAuthStore } from '@stores/authStore';
import { supabase } from '@services/supabase';

```

```

const queryClient = new QueryClient({
  defaultOptions: {
    queries: { staleTime: 1000 * 60 * 5 }, // 5 minuten cache
  },
});

export default function RootLayout() {
  const { session, setSession } = useAuthStore();
  const router = useRouter();
  const segments = useSegments();

  useEffect(() => {
    const { data: { subscription } } = supabase.auth.onAuthStateChange(
      (_event, session) => setSession(session)
    );
    return () => subscription.unsubscribe();
  }, []);

  useEffect(() => {
    const inAuthGroup = segments[0] === '(auth)';
    if (!session && !inAuthGroup) router.replace('/(auth)/login');
    if (session && inAuthGroup) router.replace('/(app)');
  }, [session, segments]);

  return (
    <QueryClientProvider client={queryClient}>
      <Slot />
    </QueryClientProvider>
  );
}

```

5.2 Supabase client + type-safe queries

src/services/supabase.ts:

```

import { createClient } from '@supabase/supabase-js';
import * as SecureStore from 'expo-secure-store';
import { Database } from '@types/database.types'; // Gegeneerd door Supabase CLI

const ExpoSecureStoreAdapter = {
  getItem: (key: string) => SecureStore.getItemAsync(key),
  setItem: (key: string, value: string) => SecureStore.setItemAsync(key, value),
  removeItem: (key: string) => SecureStore.deleteItemAsync(key),
};

export const supabase = createClient<Database>(
  process.env.EXPO_PUBLIC_SUPABASE_URL!,
  process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!,
  {
    auth: {

```

```

    storage: ExpoSecureStoreAdapter,
    autoRefreshToken: true,
    persistSession: true,
    detectSessionInUrl: false,
  },
}
);

```

Type-generatie (na elke schema-wijziging uitvoeren):

```

npx supabase gen types typescript --project-id <project-id> >
src/types/database.types.ts

```

5.3 Hooks patroon

src/hooks/useTasks.ts:

```

import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { supabase } from '@services/supabase';

export function useTodayTasks() {
  return useQuery({
    queryKey: ['tasks', 'today'],
    queryFn: async () => {
      const today = new Date().toISOString().split('T')[0];
      const { data, error } = await supabase
        .from('user_tasks')
        .select(`
          *,
          task_template:task_templates(*),
          last_completion:task_completions(completed_at)
        `)
        .eq('is_active', true)
        .lte('next_due_date', today);

      if (error) throw error;
      return data;
    },
  });
}

export function useCompleteTask() {
  const queryClient = useQueryClient();

  return useMutation({
    mutationFn: async (taskId: string) => {
      const { error } = await supabase
        .from('task_completions')
        .insert({ task_id: taskId });
    },
  });
}

```

```

    if (error) throw error;
  },
  onMutate: async (taskId) => {
    // Optimistic update: verwijder taak direct uit "vandaag" lijst
    await queryClient.cancelQueries({ queryKey: ['tasks', 'today'] });
    const previous = queryClient.getQueryData(['tasks', 'today']);
    queryClient.setQueryData(['tasks', 'today'], (old: any[]) =>
      old.filter(t => t.id !== taskId)
    );
    return { previous };
  },
  onError: (_err, _id, context) => {
    // Terugdraaien bij fout
    queryClient.setQueryData(['tasks', 'today'], context?.previous);
  },
  onSettled: () => {
    queryClient.invalidateQueries({ queryKey: ['tasks'] });
  },
});
}

```

5.4 Preset-taken seed script

supabase/seed.sql (uitgevoerd bij database initialisatie):

```

INSERT INTO task_templates (name, category, default_frequency_type,
default_interval_days, is_premium, icon) VALUES
-- Gratis taken
('Stofzuigen', 'reinigen', 'weekly', 7, false, 'vacuum'),
('Badkamer schoonmaken', 'reinigen', 'weekly', 7, false, 'shower'),
('WC schoonmaken', 'reinigen', 'weekly', 7, false, 'toilet'),
('Aanrecht schoonmaken', 'reinigen', 'daily', 1, false, 'chef-hat'),
('Vloer dweilen', 'reinigen', 'biweekly', 14, false, 'droplets'),
('Stof afnemen', 'reinigen', 'biweekly', 14, false, 'feather'),
('Was doen', 'huishouden', 'weekly', 7, false, 'shirt'),
('Vaatwasser uitruimen', 'huishouden', 'daily', 1, false, 'utensils'),

-- Premium taken
('Kliko buiten zetten', 'buiten', 'weekly', 7, true, 'trash-2'),
('Planten water geven', 'tuin', 'interval', 4, true, 'flower-2'),
('Ramen zemen', 'reinigen', 'seasonal', 90, true, 'layout'),
('Verwarmingsfilter', 'onderhoud', 'biannual', 180, true, 'thermometer'),
('Auto APK', 'administratie', 'annual', 365, true, 'car'),
('Koelkast schoonmaken', 'reinigen', 'monthly', 30, true, 'package'),
('Planten verpotten', 'tuin', 'annual', 365, true, 'shovel'),
('Grofvuil aanmelden', 'administratie', 'monthly', 30, true, 'calendar');

```

5.5 Onboarding flow

Bij eerste login: scherm met preset-taken in twee kolommen. Gratis taken zijn aanklikbaar voor iedereen. Premium taken zijn zichtbaar maar gegreyd out met een  icoon en "Premium" badge. Gebruiker selecteert zijn startset. Na selectie worden `user_tasks` records aangemaakt met de standaard frequentie en een `next_due_date` berekend op basis van vandaag.

5.6 Vandaag-scherm logica

"Vandaag" toont taken waarvan `next_due_date <= today`. Sortering: meest urgente (langst achterstallig) bovenaan. Taken die "morgen" vervallen komen in een "Binnenkort" sectie eronder. Elke taakkaart toont: naam, categorie-icoon, hoe lang geleden het voor het laatst gedaan is ("8 dagen geleden"), en een vinkje-knop. Bij afvinken: optimistic update + confetti-animatie voor engagement.

6. Fase 3 – Notificatiesysteem (week 6–7)

6.1 Push token registratie

`src/services/notifications.ts`:

```
import * as Notifications from 'expo-notifications';
import * as Device from 'expo-device';
import { supabase } from './supabase';

Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: true,
  }),
});

export async function registerPushToken(userId: string) {
  if (!Device.isDevice) return; // Werkt niet op simulator

  const { status: existingStatus } = await Notifications.getPermissionsAsync();
  let finalStatus = existingStatus;

  if (existingStatus !== 'granted') {
    const { status } = await Notifications.requestPermissionsAsync();
    finalStatus = status;
  }

  if (finalStatus !== 'granted') return;

  const token = (await Notifications.getExpoPushTokenAsync({
    projectId: process.env.EXPO_PUBLIC_EAS_PROJECT_ID,
  })).data;

  // Sla op in Supabase (upsert zodat het bij elke app-open bijgewerkt wordt)
  await supabase
    .from('push_tokens')
```

```
.upsert({ user_id: userId, token, platform: Platform.OS }, { onConflict:
'user_id,platform' });
}
```

6.2 Supabase Edge Function: dagelijkse notificaties

supabase/functions/send-daily-notifications/index.ts:

```
import { createClient } from 'https://esm.sh/@supabase/supabase-js@2';

const EXPO_PUSH_URL = 'https://exp.host/--/api/v2/push/send';

Deno.serve(async (req) => {
  // Alleen aanroepbaar via cron (check authorization header)
  const authHeader = req.headers.get('Authorization');
  if (authHeader !== `Bearer ${Deno.env.get('CRON_SECRET')}`) {
    return new Response('Unauthorized', { status: 401 });
  }

  const supabase = createClient(
    Deno.env.get('SUPABASE_URL')!,
    Deno.env.get('SUPABASE_SERVICE_ROLE_KEY')!, // Service role: mag alles lezen
  );

  const today = new Date().toISOString().split('T')[0];
  const tomorrow = new Date(Date.now() + 86400000).toISOString().split('T')[0];

  // Haal taken op die vandaag vervallen + bijbehorende push tokens
  const { data: dueTasks } = await supabase
    .from('user_tasks')
    .select(`
      id, notification_time,
      task_template:task_templates(name, is_premium),
      user:profiles(id),
      push_token:push_tokens(token)
    `)
    .eq('is_active', true)
    .lte('next_due_date', today)
    .not('push_token', 'is', null);

  // Kliko-specifieke logica: ook morgen-reminder sturen
  const { data: tomorrowKliko } = await supabase
    .from('user_tasks')
    .select(`id, task_template:task_templates(name),
push_token:push_tokens(token)`)
    .eq('task_templates.name', 'Kliko buiten zetten')
    .eq('next_due_date', tomorrow)
    .not('push_token', 'is', null);

  const messages = [
    ...(dueTasks ?? []).map(task => ({
```

```

    to: task.push_token.token,
    title: '📅 Taak herinnering',
    body: `Vergeet niet: ${task.task_template.name}`,
    data: { taskId: task.id },
  )),
  ...(tomorrowKliko ?? []).map(task => ({
    to: task.push_token.token,
    title: '🗑️ Morgen: kliko buiten!',
    body: 'Zet morgen vóór 10:00 de kliko op straat.',
    data: { taskId: task.id },
  })),
];

// Expo Push API accepteert max 100 berichten per batch
const batches = [];
for (let i = 0; i < messages.length; i += 100) {
  batches.push(messages.slice(i, i + 100));
}

for (const batch of batches) {
  await fetch(EXPO_PUSH_URL, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(batch),
  });
}

return new Response(JSON.stringify({ sent: messages.length }), { status: 200 });
});

```

6.3 Cron instellen via Supabase

In het Supabase dashboard → Edge Functions → Schedules:

```

Functie: send-daily-notifications
Schedule: 0 7 * * * (elke dag om 07:00 UTC = 08:00 of 09:00 NL)

```

Het `CRON_SECRET` sla je op via Supabase Vault:

```

supabase secrets set CRON_SECRET="jouw-willekeurige-lange-string"
supabase secrets set EXPO_PUSH_ACCESS_TOKEN="..."

```

6.4 next_due_date automatisch bijwerken

Een database trigger herberekent `next_due_date` na elke voltooiing:

```

CREATE OR REPLACE FUNCTION update_next_due_date()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE user_tasks
    SET next_due_date = CASE
        WHEN frequency_type = 'daily' THEN CURRENT_DATE + interval '1 day'
        WHEN frequency_type = 'weekly' THEN CURRENT_DATE + interval '7 days'
        WHEN frequency_type = 'biweekly' THEN CURRENT_DATE + interval '14 days'
        WHEN frequency_type = 'monthly' THEN CURRENT_DATE + interval '30 days'
        WHEN frequency_type = 'interval' THEN CURRENT_DATE + (interval_days || '
days')::interval
        ELSE CURRENT_DATE + interval '7 days'
    END,
    streak_count = streak_count + 1
    WHERE id = NEW.task_id;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER after_task_completion
AFTER INSERT ON task_completions
FOR EACH ROW EXECUTE FUNCTION update_next_due_date();

```

6b. Fase 3b – Nederlandse data-integraties (week 7–8)

6b.1 Afvalkalender-service

De afvalkalender-logica draait volledig server-side in een Supabase Edge Function. De app zelf weet niets van de afval-API — die haalt alleen de opgeslagen ophaaldata op uit de `waste_schedule` tabel.

`supabase/functions/sync-waste-schedule/index.ts`:

```

const MIJNAFVALWIJZER_KEY = Deno.env.get('MIJNAFVALWIJZER_API_KEY')!;

async function fetchWasteSchedule(postcode: string, huisnummer: string) {
    // Probeer eerst mijnafvalwijzer (dekt meeste gemeenten)
    const url = `https://api.mijnafvalwijzer.nl/webservices/appsinput/` +
        `?apikey=${MIJNAFVALWIJZER_KEY}` +
        `&method=postcodecheck` +
        `&postcode=${postcode.replace(' ', '')}` +
        `&huisnummer=${huisnummer}` +
        `&app_name=afvalwijzer&platform=phone&mobiletype=android&langs=nl`;

    const res = await fetch(url);
    const data = await res.json();

    if (data?.status === 'OK') {
        return { provider: 'mijnafvalwijzer', pickups: data.data.ophaaldagen };
    }
}

```



```

    }

    // Fallback: Ximmio (AVRI-regio)
    const ximmioUrl = `https://wasteportal.ximmio.com/api/v1/waste-calendar` +
      `?companyCode=78cd4156-394b-413d-8936-d407e334559a` +
      `&postcode=${postcode}&houseNumber=${huisnummer}`;

    const ximmioRes = await fetch(ximmioUrl);
    const ximmioData = await ximmioRes.json();
    return { provider: 'ximmio', pickups: ximmioData };
  }

Deno.serve(async (req) => {
  const supabase = createClient(/* service role */);

  // Haal alle users op met een opgeslagen adres
  const { data: addresses } = await supabase
    .from('user_addresses')
    .select('user_id, postcode, huisnummer');

  for (const addr of addresses ?? []) {
    try {
      const schedule = await fetchWasteSchedule(addr.postcode, addr.huisnummer);

      // Sla de komende 90 dagen op
      await supabase.from('waste_schedule').upsert(
        schedule.pickups.map((p: any) => ({
          user_id: addr.user_id,
          waste_type: p.type, // 'restafval' | 'gft' | 'papier' | 'pmd'
          pickup_date: p.date,
          provider: schedule.provider,
        })),
        { onConflict: 'user_id,waste_type,pickup_date' }
      );
    } catch (err) {
      // Log maar crash niet – fallback is handmatige dag instelling
      console.error(`Waste sync failed for ${addr.postcode}: ${err}`);
    }
  }

  return new Response('OK');
});

```

Cron: `0 3 * * 1` (elke maandagochtend om 03:00 — weekdata vernieuwen).

Onboarding flow voor afvalkalender:

1. Gebruiker tikt "Afvalkalender koppelen"
2. Invoer postcode + huisnummer
3. App toont preview: "We vonden: restafval elke 2 weken, GFT wekelijks, papier maandelijks"
4. Gebruiker bevestigt → `user_addresses` record aangemaakt → sync getriggerd
5. Als postcode niet gevonden: optie om handmatig ophaaldag in te stellen

Notificatie-types voor afval:

- **Avond-reminder (20:00):** "Morgen wordt je [groene/grijze/blauwe] bak geleegd. Zet hem vanavond nog buiten!"
- **Ochtend-reminder (07:00):** "Vergeet je [type] bak niet! Ophaal vandaag."
- Gebruiker kiest zelf welk type reminder (of beide).

6b.2 Weer-intelligente taakplanning

supabase/functions/weather-task-advisor/index.ts:

```
interface WeatherDay {
  date: string;
  precipitation_sum: number; // mm
  wind_speed_max: number;    // km/h
  temperature_max: number;    // °C
  sunshine_duration: number; // seconden
}

async function getWeekForecast(lat: number, lon: number): Promise<WeatherDay[]> {
  const url = `https://api.open-meteo.com/v1/forecast` +
    `?latitude=${lat}&longitude=${lon}` +
    `&daily=precipitation_sum,wind_speed_10m_max,temperature_2m_max,sunshine_duration` +
    `&timezone=Europe/Amsterdam&forecast_days=7`;

  const res = await fetch(url);
  const data = await res.json();

  return data.daily.time.map((date: string, i: number) => ({
    date,
    precipitation_sum: data.daily.precipitation_sum[i],
    wind_speed_max: data.daily.wind_speed_10m_max[i],
    temperature_max: data.daily.temperature_2m_max[i],
    sunshine_duration: data.daily.sunshine_duration[i],
  }));
}

function isGoodForOutdoorTask(day: WeatherDay): boolean {
  return (
    day.precipitation_sum < 2 && // Minder dan 2mm regen
    day.wind_speed_max < 50 && // Minder dan windkracht 7
    day.temperature_max > 8 && // Niet te koud voor ramen zemen
    day.sunshine_duration > 14400 // Meer dan 4 uur zon
  );
}

function buildWeatherAdvice(forecast: WeatherDay[], taskName: string): string {
  const goodDays = forecast.filter(isGoodForOutdoorTask);
  const dayNames = ['zondag', 'maandag', 'dinsdag', 'woensdag',
    'donderdag', 'vrijdag', 'zaterdag'];
}
```

```

if (goodDays.length === 0) {
  return `Het weer is de komende week niet ideaal voor ${taskName}. ` +
    `We herinneren je volgende week opnieuw.`;
}

const best = goodDays[0];
const dayName = dayNames[new Date(best.date).getDay()];
const others = goodDays.slice(1, 3).map(d =>
  dayNames[new Date(d.date).getDay()])
  .join(' en ');

return `Goed moment voor ${taskName}! ` +
  `${dayName.charAt(0).toUpperCase() + dayName.slice(1)} ` +
  `wordt het droog en zonnig` +
  (others ? ` . Ook ${others} zijn goede opties. ` : '.') +
  ` Regendagen deze week: ` +
  forecast.filter(d => d.precipitation_sum > 5)
    .map(d => dayNames[new Date(d.date).getDay()]).join(', ') || 'geen' + '.';
}

```

Weer-bewuste notificatie (voorbeeld output):

☐ **Ramen zemen?** Dinsdag en donderdag wordt het droog en zonnig (18°C, minimale wind).
 Woensdag verwachten we 9mm regen — dat sla je beter over.

Welke taken krijgen weers-intelligentie:

- Ramen zemen (droog + zon + geen sterke wind)
- Tuin bijhouden (droog + niet te koud)
- Auto wassen (droog + zon)
- Fiets onderhouden (buiten, droog)
- Dakgoot reinigen (droog)

Binnen-taken (stofzuigen, badkamer) krijgen nooit weers-logica — dat zou raar zijn.

6b.3 Gemeente-detectie bij onboarding

Bij het invoeren van de postcode toont de app automatisch:

- Naam van de gemeente
- Welke afvalstromen er zijn (restafval, GFT, papier, PMD, grofvuil)
- Ophaalfrequentie per stroom

Dit maakt het meteen concreet en vertrouwd voor de gebruiker. Geen abstracte "kliko buiten zetten" task, maar "Grijze bak (restafval) — elke 2 weken, volgende keer vrijdag 23 mei."

7. Fase 4 – Multi-user & huishoudens (week 8–10)

Premium feature — alleen beschikbaar voor premium abonnees.

7.1 Huishouden aanmaken & uitnodigen

Stroom:

1. Gebruiker tikt "Huishouden aanmaken" → geeft het een naam
2. Systeem genereert een 6-karakter alfanumerieke invite code (bijv. XK9P2M)
3. Code is 48 uur geldig
4. Andere gebruiker tikt "Huishouden joinen" → voert code in
5. Beide zien nu elkaars taken en voltooïingen

```
// src/services/household.ts
export async function createHousehold(name: string, userId: string) {
  const inviteCode = Math.random().toString(36).substring(2, 8).toUpperCase();
  const expiresAt = new Date(Date.now() + 48 * 60 * 60 * 1000).toISOString();

  const { data: household } = await supabase
    .from('households')
    .insert({ name, invite_code: inviteCode, invite_expires_at: expiresAt })
    .select()
    .single();

  await supabase
    .from('household_members')
    .insert({ household_id: household.id, user_id: userId, role: 'owner' });

  return household;
}

export async function joinHousehold(code: string, userId: string) {
  const { data: household } = await supabase
    .from('households')
    .select('id, invite_expires_at')
    .eq('invite_code', code.toUpperCase())
    .single();

  if (!household) throw new Error('Ongeldige code');
  if (new Date(household.invite_expires_at) < new Date()) throw new Error('Code verlopen');

  await supabase
    .from('household_members')
    .insert({ household_id: household.id, user_id: userId, role: 'member' });
}
```

7.2 Realtime sync voor gedeelde taken

Wanneer een huisgenoot een taak afvinkt, zien alle bewoners het direct:

```
// In de Vandaag-component
useEffect(() => {
  if (!householdId) return;
```

```
const channel = supabase
  .channel(`household:${householdId}`)
  .on('postgres_changes',
    { event: 'INSERT', schema: 'public', table: 'task_completions' },
    (payload) => {
      queryClient.invalidateQueries({ queryKey: ['tasks'] });
      // Optioneel: toast tonen "Jasper heeft de badkamer schoongemaakt! 🧹"
    }
  )
  .subscribe();

return () => { supabase.removeChannel(channel); };
}, [householdId]);
```

7.3 RLS policies voor huishoudens

```
-- Gebruikers zien alleen taken van hun eigen huishouden
CREATE POLICY "household_tasks_select" ON user_tasks
FOR SELECT USING (
  household_id IN (
    SELECT household_id FROM household_members WHERE user_id = auth.uid()
  )
);

-- Alleen owners kunnen leden verwijderen
CREATE POLICY "remove_member_owner_only" ON household_members
FOR DELETE USING (
  EXISTS (
    SELECT 1 FROM household_members
    WHERE household_id = household_members.household_id
    AND user_id = auth.uid()
    AND role = 'owner'
  )
);
```

8. Fase 5 – Monetisatie (week 11–12)

8.1 RevenueCat setup

RevenueCat is de standaard voor in-app purchases in React Native. Het handelt de complexiteit van Apple IAP en Google Play Billing af, inclusief receipt validation en webhook-integratie.

```
npm install react-native-purchases
```

[src/services/revenuecat.ts](#):

```
import Purchases, { LOG_LEVEL } from 'react-native-purchases';

export function initRevenueCat(userId: string) {
  if (__DEV__) Purchases.setLogLevel(LOG_LEVEL.DEBUG);

  Purchases.configure({
    apiKey: Platform.select({
      ios: process.env.EXPO_PUBLIC_REVENUECAT_IOS_KEY!,
      android: process.env.EXPO_PUBLIC_REVENUECAT_ANDROID_KEY!,
    })!,
    appUserID: userId, // Koppel aan Supabase user ID
  });
}

export async function getSubscriptionStatus() {
  const customerInfo = await Purchases.getCustomerInfo();
  return customerInfo.entitlements.active['premium'] !== undefined;
}

export async function purchasePremium() {
  const offerings = await Purchases.getOfferings();
  const monthly = offerings.current?.monthly;
  if (!monthly) throw new Error('Geen aanbod beschikbaar');
  return Purchases.purchasePackage(monthly);
}
```

8.2 Webhook: RevenueCat → Supabase

RevenueCat stuurt webhooks bij aankoop, verlenging, annulering, etc. Een Edge Function verwerkt deze en updatet je eigen `subscriptions` tabel. Dit is nodig zodat je server-side (in RLS policies en Edge Functions) de subscriptiestatus kan controleren.

`supabase/functions/revenuecat-webhook/index.ts`:

```
Deno.serve(async (req) => {
  const payload = await req.json();
  const supabase = createClient(/* service role */);

  const { event } = payload;
  const userId = event.app_user_id; // Matcht je Supabase user ID
  const isActive = ['INITIAL_PURCHASE', 'RENEWAL',
    'UNCANCELLATION'].includes(event.type);

  await supabase
    .from('subscriptions')
    .upsert({
      user_id: userId,
      is_premium: isActive,
      expires_at: event.expiration_at_ms
        ? new Date(event.expiration_at_ms).toISOString()
    })
  });
```

```

      : null,
      product_id: event.product_id,
    }, { onConflict: 'user_id' });

    return new Response('OK', { status: 200 });
  });

```

Webhook URL instellen in RevenueCat dashboard:

<https://<project>.supabase.co/functions/v1/revenuecat-webhook>

8.3 Premium gate component

`src/components/PremiumGate.tsx`:

```

import { useSubscription } from '@/hooks/useSubscription';
import { router } from 'expo-router';

interface Props {
  children: React.ReactNode;
  feature?: string; // Voor analytics
}

export function PremiumGate({ children, feature }: Props) {
  const { isPremium, isLoading } = useSubscription();

  if (isLoading) return null;

  if (!isPremium) {
    return (
      <TouchableOpacity
        onPress={() => {
          analytics.track('premium_gate_tapped', { feature });
          router.push('/(app)/settings/subscription');
        }}
      >
        {children}
        <View style={styles.overlay}>
          <Text>🔒 Premium</Text>
        </View>
      </TouchableOpacity>
    );
  }

  return <>{children}</>;
}

```

8.4 Free tier limiet op custom taken

RLS policy die custom taken beperkt:

```
-- Gratis gebruikers: max 3 custom taken
CREATE POLICY "custom_task_limit_free" ON user_tasks
  FOR INSERT WITH CHECK (
    -- Premium? Altijd ok
    EXISTS (SELECT 1 FROM subscriptions WHERE user_id = auth.uid() AND is_premium
= true)
    OR
    -- Gratis: tel bestaande custom taken
    (SELECT COUNT(*) FROM user_tasks WHERE user_id = auth.uid() AND is_custom =
true) < 3
  );
```

8.5 Upgradescherm

Het upgradescherm toont een duidelijke waardepropositie:

- Lijst van premium-taken die ze zullen ontgrendelen (specifiek: "Kliko buiten zetten", "Planten water geven", etc.)
- Prijs: €3,49/maand of €27,99/jaar (bespaar 33%)
- Social proof: "X andere huishoudens gebruiken al de premium taken"
- Gratis proefperiode: 7 dagen (via RevenueCat introductory offer)

Apple- en Google-richtlijn: het upgradeproces moet volledig via de native store dialoog verlopen. Verwijs nooit naar andere betaalmethodes in de app zelf.

9. Fase 6 – Release prep (week 13–16)

9.1 App Store & Play Store assets

App-icoon: 1024×1024px PNG, geen afgeronde hoeken (iOS doet dat zelf), geen transparantie. Gebruik een simpel, herkenbaar beeld — bijv. een vinkje in een schoon huis.

Screenshots:

- iOS: 6.7 inch (iPhone 15 Pro Max) + 12.9 inch iPad
- Android: 16:9 en 9:16
- Toon de "Vandaag" scherm, het voltooiingsmoment (vinkje + animatie), en de preset-picker

App Store tekst (NL):

Titel: TaskThuis – Huishoudplanner
Ondertitel: Nooit meer vergeten schoon te maken

Beschrijving:

Vergeet de badkamer, kliko of planten nooit meer. TaskThuis helpt je alle terugkerende huishoudtaken bij te houden met slimme herinneringen op het juiste moment.

- Vink taken af en krijg automatisch de volgende herinnering gepland
- Push-notificaties wanneer het tijd is – niet eerder, niet later
- Deel taken met huisgenoten en zie wie wat heeft gedaan
- Kies uit een uitgebreide lijst met standaard huishoudtaken

Perfect voor iedereen die voor het eerst op zichzelf woont.

9.2 Privacy & GDPR

Privacy policy is verplicht voor beide stores. Minimale vereisten voor jouw app:

- Welke data je verzamelt (email, naam, taakdata, push token)
- Waarvoor (app-functionaliteit, notificaties)
- Hoe lang je het bewaart
- Recht op inzage, correctie en verwijdering

Data verwijdering (verplicht voor App Store):

```
// settings/account.tsx
async function deleteAccount(userId: string) {
  // 1. Verwijder alle persoonlijke data
  await supabase.from('task_completions').delete().eq('completed_by', userId);
  await supabase.from('user_tasks').delete().eq('user_id', userId);
  await supabase.from('push_tokens').delete().eq('user_id', userId);
  await supabase.from('profiles').delete().eq('id', userId);
  // 2. Verwijder auth account (via service role in edge function)
  await supabase.functions.invoke('delete-user', { body: { userId } });
}
```

9.3 Security audit voor release

Voer dit uit vóór release:

```
# Dependency vulnerabilities
npm audit --audit-level=high

# Check of er secrets in je code staan
npx secretlint "**/*.ts" "**/*.tsx"

# Bundle analyse (geen onnodige packages)
npx expo export && npx source-map-explorer
```

Supabase RLS checklist:

- Test elke tabel met een andere user_id in Postman (met de anon key)
- Verifieer dat je nooit de service role key kunt ophalen vanuit de app
- Check dat `task_templates` alleen leesbaar is, niet schrijfbaar door gebruikers

- Verifieer dat Edge Functions alleen reageren op `CRON_SECRET`

9.4 EAS Build + Submit

```
# Productie build
eas build --profile production --platform all

# Na goedkeuring intern testen via TestFlight en Google Play Internal:
eas submit --profile production --platform ios
eas submit --profile production --platform android
```

9.5 Post-release monitoring

Sentry alert instellen voor:

- Error rate > 1% in een tijdsspanne van 1 uur
- Crash-free rate < 99%
- Edge function failures (notificaties die niet verstuurd worden)

PostHog events die je wilt tracken:

- `task_completed` (taak afgevinkt)
- `task_added` (nieuwe taak aangemaakt)
- `premium_gate_tapped` (vrije gebruiker raakt een premium feature)
- `subscription_started` (aankoop voltooid)
- `notification_permission_granted` / `denied`

10. Veiligheidschecklist

Keys & secrets

- ☐ `.env.local` staat in `.gitignore` en is nooit gecommit
- ☐ Supabase **anon key** is veilig om in de app te gebruiken (RLS beschermt het)
- ☐ Supabase **service role key** staat alleen in EAS Secrets en Supabase Vault — nooit in de app
- ☐ RevenueCat keys staan in EAS Secrets
- ☐ `CRON_SECRET` voor Edge Functions staat in Supabase Vault
- ☐ Expo Push Access Token staat in EAS Secrets

Database beveiliging

- ☐ RLS is ingeschakeld op **elke** tabel
- ☐ Elke policy is getest met een andere user's JWT token
- ☐ `task_templates` is read-only voor authenticated users
- ☐ `subscriptions` is alleen schrijfbaar via de webhook Edge Function (service role)
- ☐ `push_tokens` is alleen leesbaar/schrijfbaar door de eigenaar

App-beveiliging

- ☐ Auth tokens opgeslagen in **expo-secure-store** (niet **AsyncStorage**)
- ☐ Alle formulierinput gevalideerd met Zod vóór API-aanroep
- ☐ Rate limiting op auth endpoints (Supabase heeft dit ingebouwd)
- ☐ Deep link validatie in expo-router (geen open redirects)
- ☐ Certificate pinning overwogen voor productie (via expo-crypto)

Privacy & compliance

- ☐ Privacy policy gepubliceerd op een toegankelijke URL
- ☐ Account-verwijdering volledig geïmplementeerd (App Store vereiste)
- ☐ Data-export optie beschikbaar (GDPR artikel 20)
- ☐ PostHog configureerd zonder IP-logging (privacy-vriendelijk)
- ☐ Geen analytics data zonder toestemming (cookie/tracking banner bij eerste gebruik)

11. Volledige mappenstructuur

```

taskapp/
├── app/                                # expo-router pagina's
│   ├── _layout.tsx
│   ├── +not-found.tsx
│   ├── (auth)/
│   │   ├── _layout.tsx
│   │   ├── login.tsx
│   │   ├── register.tsx
│   │   └── onboarding.tsx
│   └── (app)/
│       ├── _layout.tsx
│       ├── index.tsx                # Vandaag
│       ├── tasks/
│       │   ├── index.tsx
│       │   ├── [id].tsx
│       │   └── add.tsx
│       └── settings/
│           ├── index.tsx
│           ├── household.tsx
│           ├── subscription.tsx
│           └── account.tsx
├── src/
│   ├── components/
│   │   ├── ui/                    # Herbruikbare atoms
│   │   │   ├── Button.tsx
│   │   │   ├── Card.tsx
│   │   │   ├── Badge.tsx
│   │   │   └── ProgressRing.tsx
│   │   └── tasks/
│   │       ├── TaskCard.tsx
│   │       ├── TaskList.tsx
│   │       ├── FrequencyPicker.tsx
│   │       └── PresetPicker.tsx

```

```

├── household/
│   ├── MemberCard.tsx
│   ├── InviteModal.tsx
│   └── PremiumGate.tsx
├── hooks/
│   ├── useTasks.ts
│   ├── useTodayTasks.ts
│   ├── useTaskCompletions.ts
│   ├── useHousehold.ts
│   └── useSubscription.ts
├── services/
│   ├── supabase.ts           # Client + typed helpers
│   ├── notifications.ts      # Token + permission management
│   ├── revenuecat.ts         # Purchase + entitlement helpers
│   └── analytics.ts          # PostHog + Sentry wrapper
├── stores/
│   ├── authStore.ts          # Session state (Zustand)
│   └── uiStore.ts            # Filter, modal state
├── constants/
│   ├── frequencies.ts        # Frequentie-opties
│   └── categories.ts         # Taakcategorieën + iconen
├── types/
│   ├── database.types.ts     # Gegenereerd door Supabase CLI
│   └── app.types.ts          # Extra app-specifieke types
├── supabase/
│   └── migrations/           # Elke schema-wijziging als genummerd .sql
bestand
│   ├── 001_initial_schema.sql
│   ├── 002_add_streaks.sql
│   └── 003_add_households.sql
│   ├── functions/
│   │   ├── send-daily-notifications/
│   │   │   └── index.ts
│   │   ├── revenuecat-webhook/
│   │   │   └── index.ts
│   │   └── delete-user/
│   │       └── index.ts
│   └── seed.sql              # Preset-taken
├── assets/
│   ├── images/
│   │   ├── icon.png         # 1024x1024
│   │   └── splash.png
│   └── fonts/
├── .github/
│   └── workflows/
│       └── ci.yml

```

```

├─ docs/
│   └─ implementatieplan.md      # Document met alle stappen om deze app te
realiseren
├─ .env.local                    # NOOIT committen
├─ .env.example                  # Wél committen (lege waarden)
├─ eas.json
├─ app.json
├─ tailwind.config.js
└─ tsconfig.json

```

12. Database schema met RLS

```

-- Profiles (uitbreiding op auth.users)
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  display_name TEXT,
  avatar_url TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
CREATE POLICY "own_profile" ON profiles USING (auth.uid() = id);

-- Subscriptions
CREATE TABLE subscriptions (
  user_id UUID PRIMARY KEY REFERENCES profiles(id) ON DELETE CASCADE,
  is_premium BOOLEAN DEFAULT FALSE,
  product_id TEXT,
  expires_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

ALTER TABLE subscriptions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "read_own_subscription" ON subscriptions FOR SELECT USING
(auth.uid() = user_id);
-- Schrijven alleen via service role (webhook)

-- Task templates (de preset bibliotheek)
CREATE TABLE task_templates (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  category TEXT NOT NULL,
  default_frequency_type TEXT NOT NULL,
  default_interval_days INTEGER DEFAULT 7,
  is_premium BOOLEAN DEFAULT FALSE,
  icon TEXT
);

ALTER TABLE task_templates ENABLE ROW LEVEL SECURITY;

```

```

CREATE POLICY "read_free_templates" ON task_templates
  FOR SELECT USING (
    is_premium = FALSE
    OR EXISTS (SELECT 1 FROM subscriptions WHERE user_id = auth.uid() AND
is_premium = TRUE)
  );

-- Households
CREATE TABLE households (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  invite_code TEXT UNIQUE,
  invite_expires_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Household members
CREATE TABLE household_members (
  household_id UUID REFERENCES households(id) ON DELETE CASCADE,
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  role TEXT DEFAULT 'member' CHECK (role IN ('owner', 'member')),
  joined_at TIMESTAMPTZ DEFAULT NOW(),
  PRIMARY KEY (household_id, user_id)
);

ALTER TABLE household_members ENABLE ROW LEVEL SECURITY;
CREATE POLICY "view_own_household" ON household_members FOR SELECT
  USING (user_id = auth.uid() OR household_id IN (
    SELECT household_id FROM household_members WHERE user_id = auth.uid()
  ));

-- User tasks
CREATE TABLE user_tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  household_id UUID REFERENCES households(id) ON DELETE SET NULL,
  template_id UUID REFERENCES task_templates(id),
  is_custom BOOLEAN DEFAULT FALSE,
  custom_name TEXT,
  frequency_type TEXT NOT NULL,
  interval_days INTEGER DEFAULT 7,
  notification_time TIME DEFAULT '09:00',
  next_due_date DATE NOT NULL,
  streak_count INTEGER DEFAULT 0,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

ALTER TABLE user_tasks ENABLE ROW LEVEL SECURITY;
CREATE POLICY "own_or_household_tasks" ON user_tasks FOR SELECT
  USING (
    user_id = auth.uid()
    OR household_id IN (
      SELECT household_id FROM household_members WHERE user_id = auth.uid()
    )
  )

```

```
)
);
CREATE POLICY "insert_own_tasks" ON user_tasks FOR INSERT
WITH CHECK (user_id = auth.uid());
CREATE POLICY "update_own_tasks" ON user_tasks FOR UPDATE
USING (user_id = auth.uid());

-- Task completions
CREATE TABLE task_completions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  task_id UUID REFERENCES user_tasks(id) ON DELETE CASCADE,
  completed_by UUID REFERENCES profiles(id),
  completed_at TIMESTAMPTZ DEFAULT NOW()
);

ALTER TABLE task_completions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "view_task_completions" ON task_completions FOR SELECT
USING (
  task_id IN (
    SELECT id FROM user_tasks WHERE user_id = auth.uid()
  )
);
CREATE POLICY "insert_completion" ON task_completions FOR INSERT
WITH CHECK (completed_by = auth.uid());

-- Push tokens
CREATE TABLE push_tokens (
  user_id UUID REFERENCES profiles(id) ON DELETE CASCADE,
  platform TEXT CHECK (platform IN ('ios', 'android')),
  token TEXT NOT NULL,
  updated_at TIMESTAMPTZ DEFAULT NOW(),
  PRIMARY KEY (user_id, platform)
);

ALTER TABLE push_tokens ENABLE ROW LEVEL SECURITY;
CREATE POLICY "own_push_tokens" ON push_tokens USING (auth.uid() = user_id);
```

Laatste update: 2026 — Gegenereerd als onderdeel van portfolio-project.